

Logic Design and Switching Theory

Lecture – 1

Subtitle: Introduction to Digital Logic

by
Ms. Hera Noor

LOs – Learning Objectives

Introduction to Digital Design Number System

- Non-positional number system
- Positional number system
- Decimal number system
- Binary number system
- Octal number system
- Hexadecimal number system

Convert a number's base

- Another base to decimal base
- Decimal base to another base

- Some base to another base

Shortcut methods for converting

- Binary to Octal
- Octal to binary
- Binary to Hexadecimal
- Hexadecimal to Binary

Background

- The term **digital design** describes a wide range of computer-related skills and includes work in areas such as web design, digital imaging, and 3D modeling.
- Digital design is a branch of graphic design that uses computers, graphics tablets, and other electronic devices to create graphics and designs for the web, television, print, and portable electronic devices.
- It is an evolving industry that is constantly changing and exploring new devices and technologies.
- Micro electronic technologies have revolutionized our world: cell phones, internet, advancements in medicine etc.
- **The semiconductor industry has grown from \$21 billion in 1985 to \$315 billion in 2013 and \$700 billion in 2025 and continues to expand rapidly.**

The Digital Revolution

Integrated Circuit: Many digital operations on the same material

Vacuum tubes



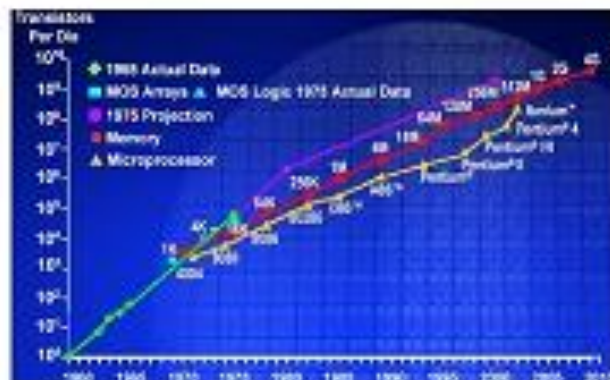
ENIAC



(1.6 x 11.1 mm)

Integrated Circuit

Exponential Growth of Computation



Moore's Law

WWII Model

1949

1965

OCT
2025

OVERVIEW OF SOCIAL MEDIA USE

HEADLINES FOR SOCIAL MEDIA ADOPTION AND USE [NOTE: USER IDENTITIES MAY NOT REPRESENT UNIQUE INDIVIDUALS]



GLOBAL OVERVIEW

NUMBER OF SOCIAL
MEDIA USER IDENTITIES



5.66
BILLION



QUARTER-ON-QUARTER CHANGE
IN SOCIAL MEDIA USER IDENTITIES



+1.1%
+62 MILLION

we
are
social

YEAR-ON-YEAR CHANGE IN
SOCIAL MEDIA USER IDENTITIES



+4.8%
+259 MILLION

Meltwater

WEEKLY TIME SPENT USING
SOCIAL PLATFORMS (INC. VIDEO)



18H 36M

GWJ.

AVERAGE NUMBER OF SOCIAL
PLATFORMS USED EACH MONTH



6.75
YOY: **-0.2% (-0.02)**

SOCIAL MEDIA
USER IDENTITIES vs.
TOTAL POPULATION



68.7%



SOCIAL MEDIA USER
IDENTITIES AGED 18+ vs.
POPULATION AGED 18+



92.6%

Meltwater

SOCIAL MEDIA USER
IDENTITIES vs. INDIVIDUALS
USING THE INTERNET



93.8%



FEMALE SOCIAL MEDIA USER
IDENTITIES vs. TOTAL SOCIAL
MEDIA USER IDENTITIES



45.6%

we
are
social

MALE SOCIAL MEDIA USER
IDENTITIES vs. TOTAL SOCIAL
MEDIA USER IDENTITIES



54.4%

Robert Noyce (1927–1990)

- Known as the “Mayor of Silicon Valley”
- Co-founded Fairchild Semiconductor in 1957
- Co-founded Intel in 1968
- Co-inventor of the integrated circuit



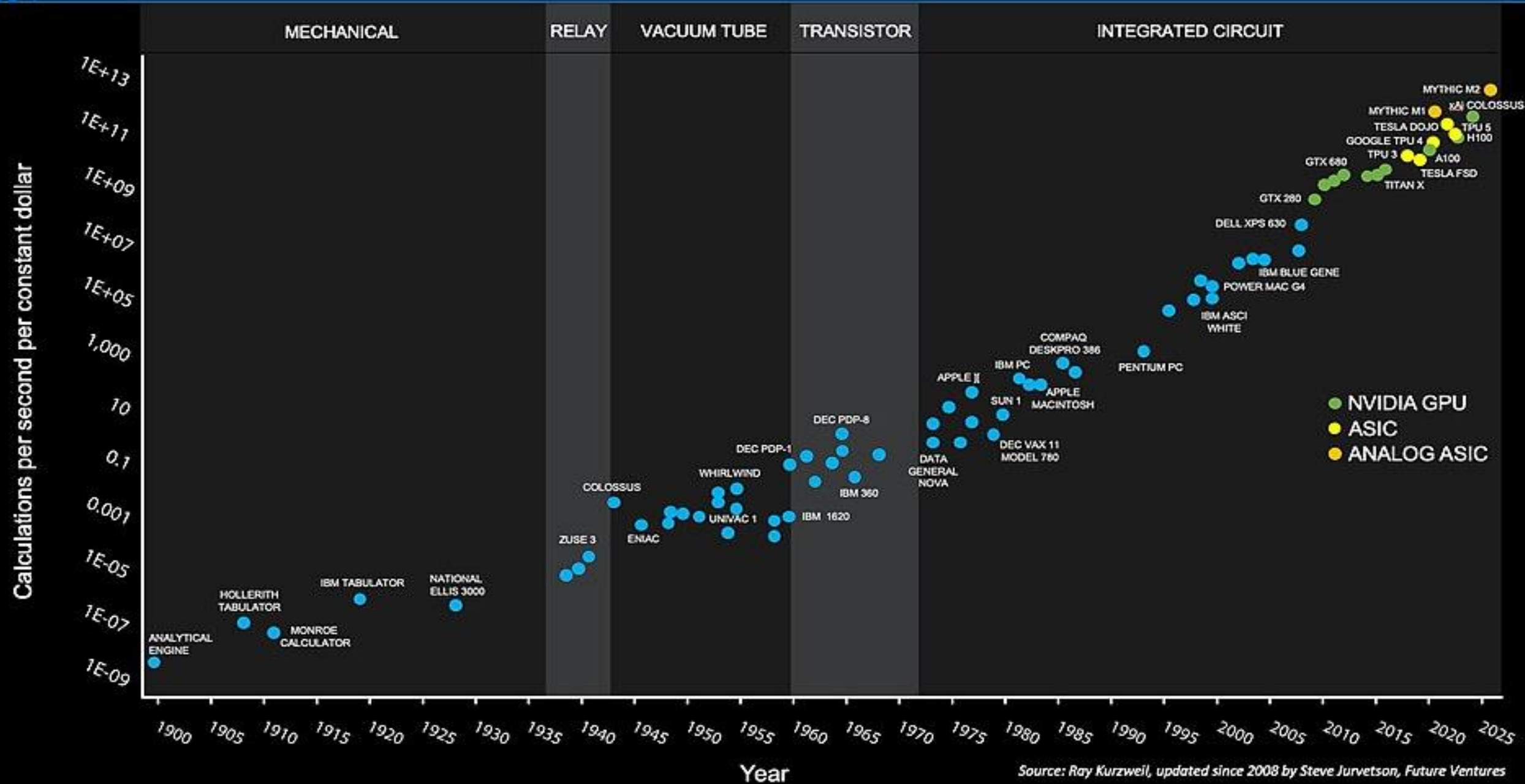
Gordon Moore



- Co-founded Intel with Robert Noyce in 1968
- Formulated Moore's Law, stating that the number of transistors on a computer chip doubles approximately every 1.5 years (observed in 1965)



128 YEARS OF MOORE'S LAW



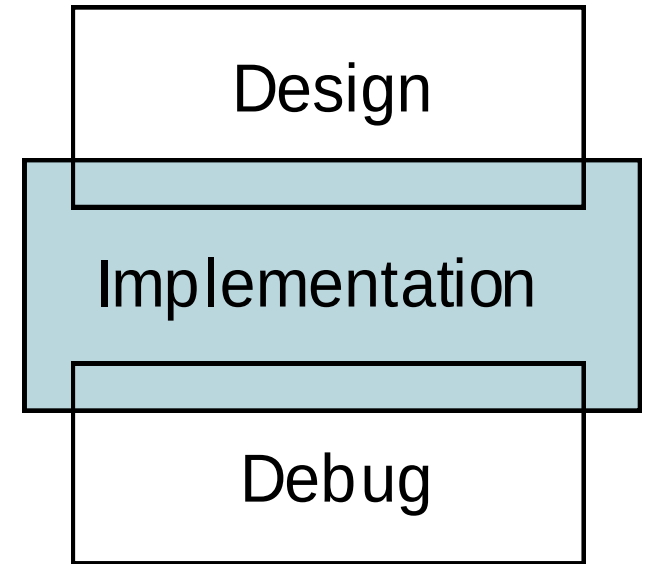
Design Process

Design: Planning how a system or solution will look and function.

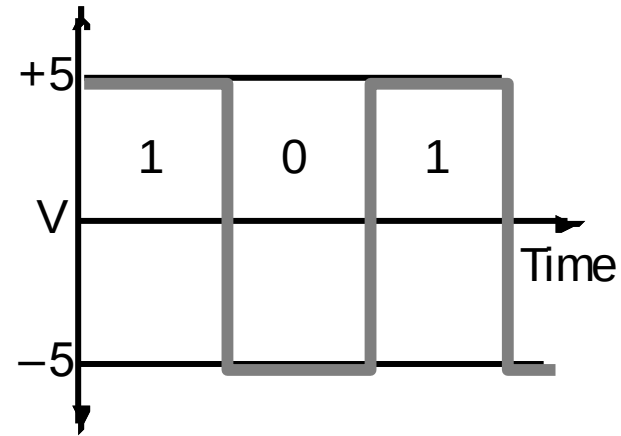
Implementation: Converting the design into a working system or product.

Debug: Finding and fixing errors to ensure correct operation.

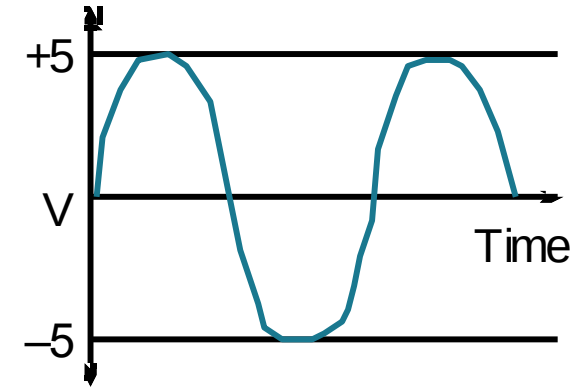
Design plans the solution, **implementation** builds it, and **debugging** finds and fixes errors to make it work correctly.



Digital vs. Analog



Digital:
only assumes discrete values



Analog:
values vary over a broad range
continuously

Number Systems

A number system is a way of writing and representing numbers using symbols according to consistent rules.

Non-positional Number Systems

Characteristics

- Use symbols such as I for 1, II for 2, III for 3, IIII for 4, IIIII for 5, etc.
- Each symbol represents the same value regardless of its position in the number.
- The symbols are simply added to find the value of a particular number.

Difficulty

- It is difficult to perform arithmetic with such a number system.

Number Systems

Positional Number Systems

Characteristics

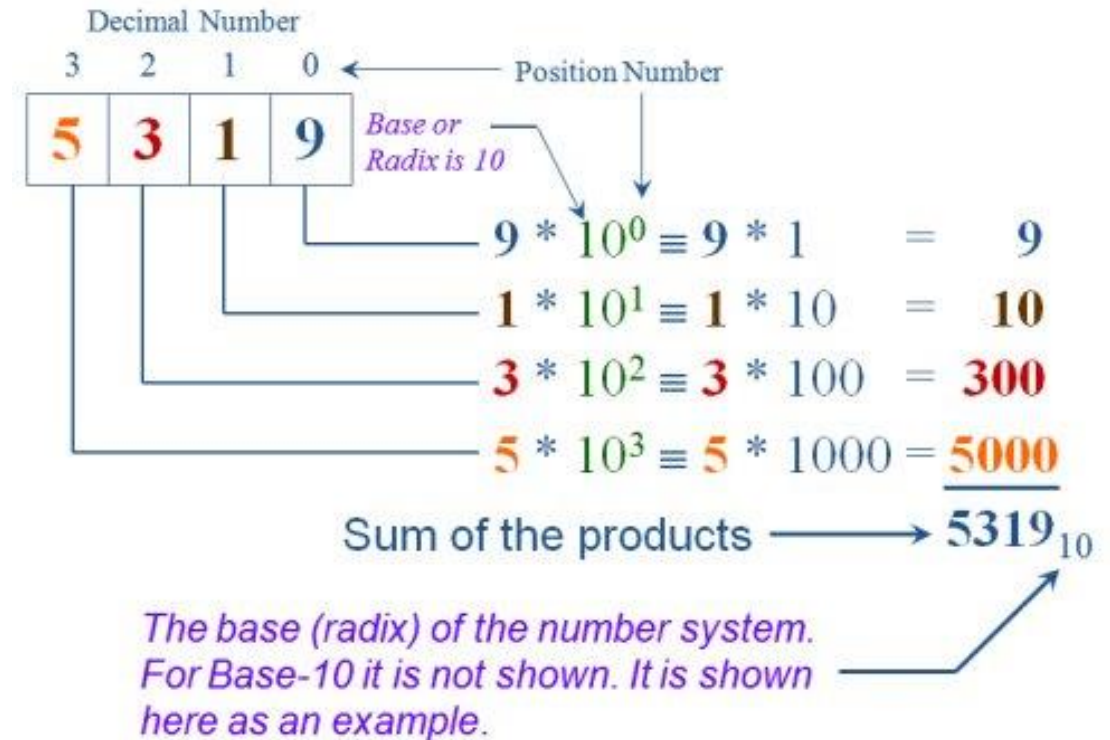
- Use only few symbols called digits.
- These symbols represent different values depending on the position they occupy in the number.
- The value of each digit depends on its position and the base (radix).

Main Positional Number Systems

- Decimal Number System (Base 10)
- Binary
- Octal
- Hexadecimal

Decimal Number System (Base 10)

- It has 10 symbols or digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9). Hence, its base = 10
- The maximum value of a single digit is 9 (one less than the value of the base)
- Each place represents powers of 10.



Binary Number System (Base 2)

Characteristics

- Uses digits 0 and 1.
- Each place represents powers of 2.
- Used by computers for internal processing.
- A positional number system
- Has only 2 symbols or digits (0 and 1).
- Hence its base = 2
- The maximum value of a single digit is 1 (one less than the value of the base)

Binary Number System (Base 2)

Decimal Number	Binary Number	Decimal Number	Binary Number
1	1	11	1011
2	10	12	1100
3	11	13	1101
4	100	14	1110
5	101	15	1111
6	110	16	10000
7	111	17	10001
8	1000	18	10010
9	1001	19	10011
10	1010	20	10100

Bits

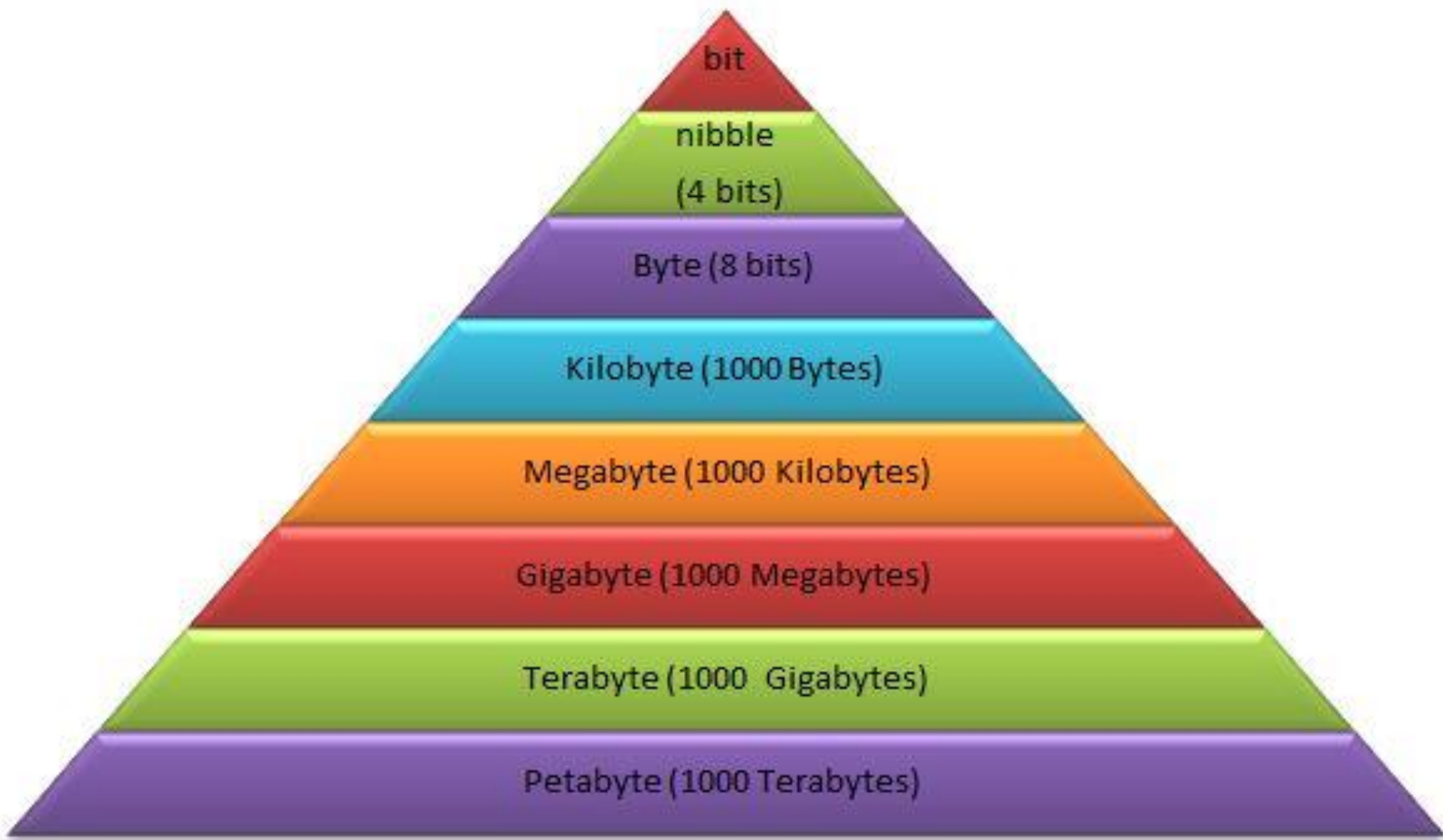
Definition: A binary digit, the most basic unit of data in computing.

Values: Can be 0 or 1, true or false, on or off.

Storage: Stored as electrical charges (high/low voltage) or magnetic states.

Units: 8 bits = 1 byte; 1000 bytes = 1 kilobyte (KB), etc..

Usage: Essential for all digital communication, programming, and storage.



bit

nibble
(4 bits)

Byte (8 bits)

Kilobyte (1000 Bytes)

Megabyte (1000 Kilobytes)

Gigabyte (1000 Megabytes)

Terabyte (1000 Gigabytes)

Petabyte (1000 Terabytes)

Octal Number System (Base 8)

Characteristics

- An octal number is a number expressed in base 8. That means it uses 8 digits, specifically: (0,1,2,3,4,5,6,7).
- Each place represents powers of 8.

Octal	Decimal	Calculation
0	0	$0 \times 8^0 = 0$
1	1	$1 \times 8^0 = 1$
2	2	$2 \times 8^0 = 2$
3	3	$3 \times 8^0 = 3$
4	4	$4 \times 8^0 = 4$
5	5	$5 \times 8^0 = 5$
6	6	$6 \times 8^0 = 6$
7	7	$7 \times 8^0 = 7$
10	8	$1 \times 8^1 + 0 \times 8^0 = 8 + 0 = 8$
11	9	$1 \times 8^1 + 1 \times 8^0 = 8 + 1 = 9$
12	10	$1 \times 8^1 + 2 \times 8^0 = 8 + 2 = 10$
13	11	$1 \times 8^1 + 3 \times 8^0 = 8 + 3 = 11$
14	12	$1 \times 8^1 + 4 \times 8^0 = 8 + 4 = 12$

Octal Number System (Base 8)

Conversion

Decimal $\rightarrow$ Octal: Divide by 8 repeatedly and write remainders backward.

Octal $\rightarrow$ Decimal: Multiply each digit by powers of 8 (as shown above).

Hexadecimal Number System (Base 16)

Characteristics

- A positional number system
- Has total 16 symbols or digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F).
Hence its base = 16
- The symbols A, B, C, D, E and F represent the decimal values 10, 11, 12, 13, 14 and 15 respectively
- The maximum value of a single digit is 15 (one less than the value of the base)

Hexadecimal Number System (Base 16)

Conversion

- Each position of a digit represents a specific power of the base (16) Since there are only 16 digits, 4 bits ($2^4 = 16$) are sufficient to represent any hexadecimal number in binary

Example

$$\begin{aligned} &1AF_{16} \\ &= (1 \times 16^2) + (A \times 16^1) + (F \times 16^0) \\ &= 1 \times 256 + 10 \times 16 + 15 \times 1 \\ &= 256 + 160 + 15 \\ &= \mathbf{431}_{10} \end{aligned}$$

Conversion from Decimal to another base

Division-Remainder Method

To convert a decimal number N into a base B (where $B \geq 2$):

- Divide the decimal number by the new base B .
- Record the remainder.
- Replace the number with the quotient from the division.
- Repeat steps 1–3 until the quotient becomes 0.
- The digits of the new base number are the remainders read in reverse order (from last to first).

Conversion from Decimal to another base

Decimal to Binary

2	278	0
2	139	1
2	69	1
2	34	0
2	17	1
2	8	0
2	4	0
2	2	0
	1	

$$(278)_{10} = (100010110)_2$$

Conversion from Decimal to another base

Decimal to Octal

Divisor	Dividend	Remainder
8	100	
8	12	4
8	1	4
	0	1

Reverse

100



144

Conversion from Decimal to another base

Decimal to Octal

Decimal Number: 2545

16	2545	1 < 10, arr[0] = 1	↑ remainder
16	159	15 > 10, arr[1] = F	
16	9	9 < 10, arr[2] = 9	
	0		

Hexadecimal number: 41

Conversion from Decimal to another base

Positional Value Method

Every number in any base can be expressed as a sum of its digits multiplied by powers of the base.

If a number in base B is written as:

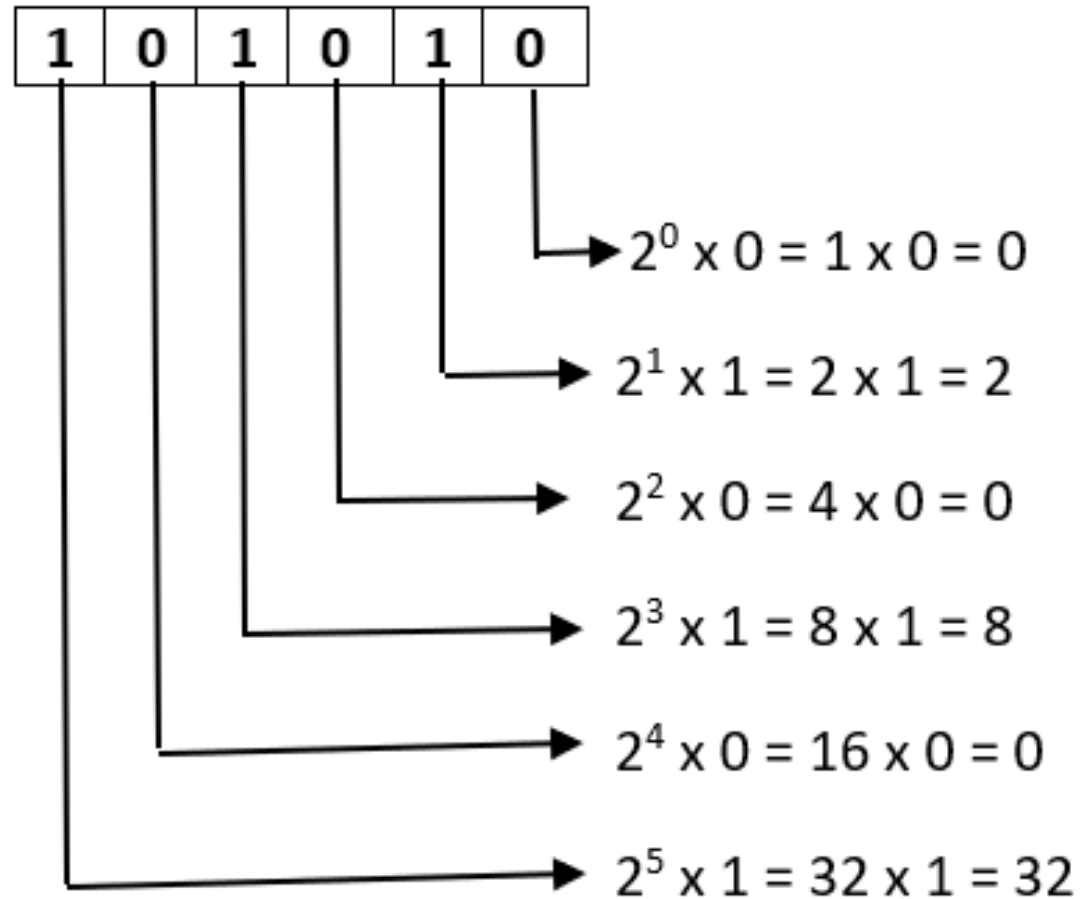
$$(d_n d_{n-1} \dots d_2 d_1 d_0)_B$$

Then its decimal equivalent is:

$$d_n \cdot B^n + d_{n-1} \cdot B^{n-1} + \dots + d_1 \cdot B^1 + d_0 \cdot B^0$$

Conversion from Decimal to another base

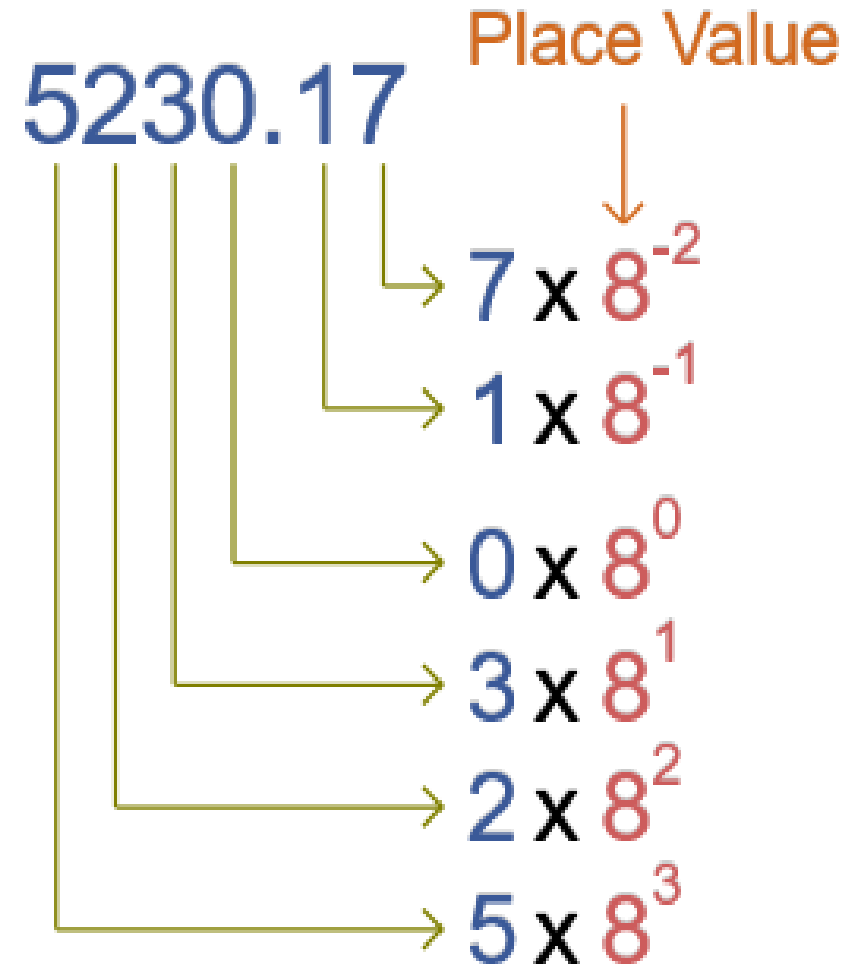
Binary to Decimal



Resultant decimal number = $0 + 2 + 0 + 8 + 0 + 32 = 42$

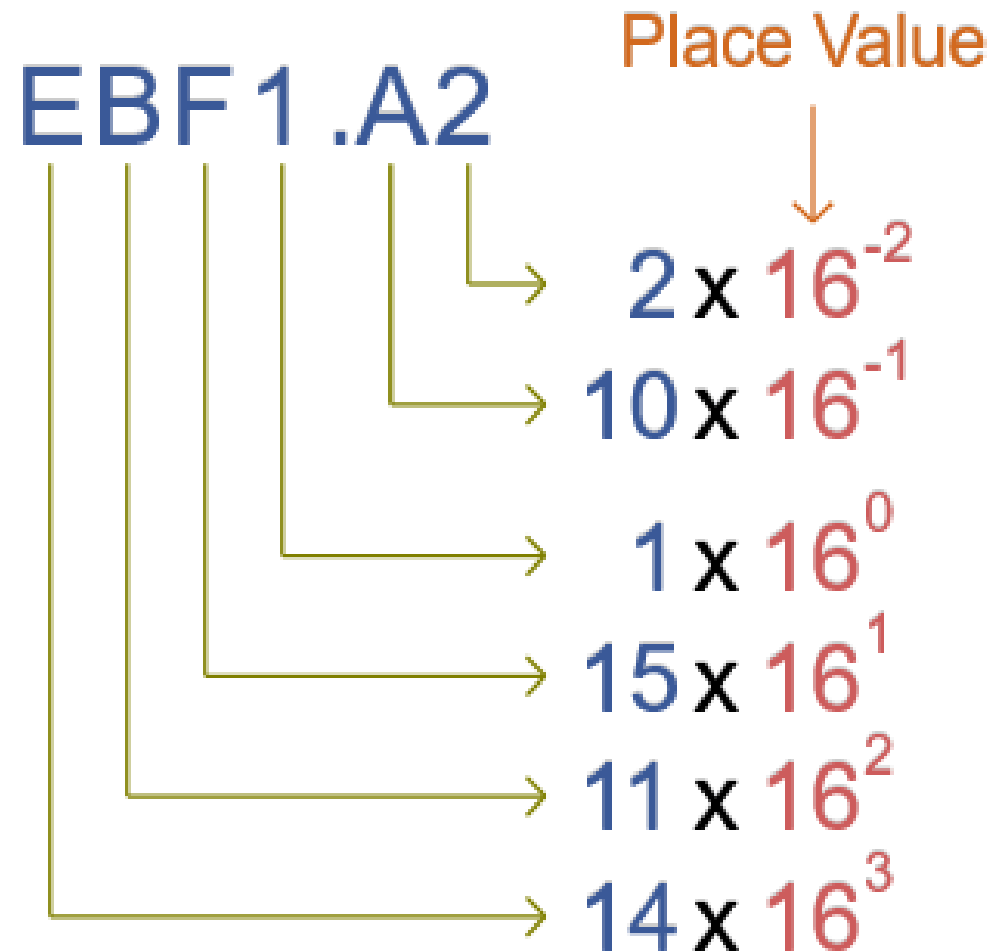
Conversion from Decimal to another base

Octal to Decimal



Conversion from Decimal to another base

Hexadecimal to Decimal



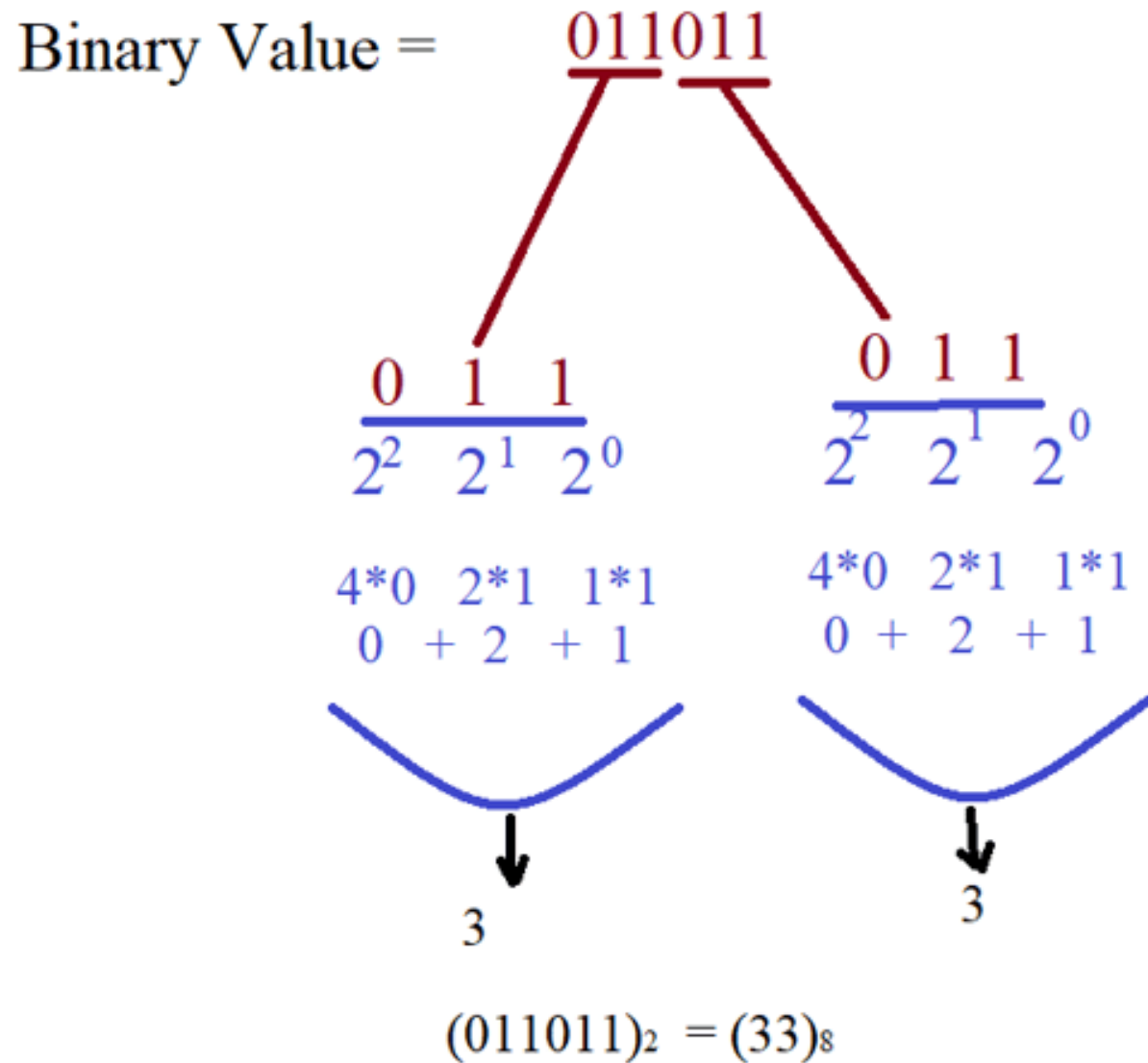
Shortcut Method for Converting a Binary Number to Octal Number

Grouping Bits

- Step 1: Write the binary number.
- Step 2: Starting from the right (least significant bit), group the binary digits in sets of 3. Add extra 0s at the leftmost side if necessary to make a complete group of 3.
- Step 3: Convert each group of 3 bits directly to its octal equivalent.
- Step 4: Combine the octal digits in the same order.

Why it works: $2^3=8$, so 3 binary bits perfectly map to 1 octal digit.

Binary to Octal



Octal to Binary

1 2 . 5
/ / \
001 010 101

$$(12.5)_8 = (001010.101)_2$$

7 2 1
/ / \
111 010 001

$$(721)_8 = (111010001)_2$$

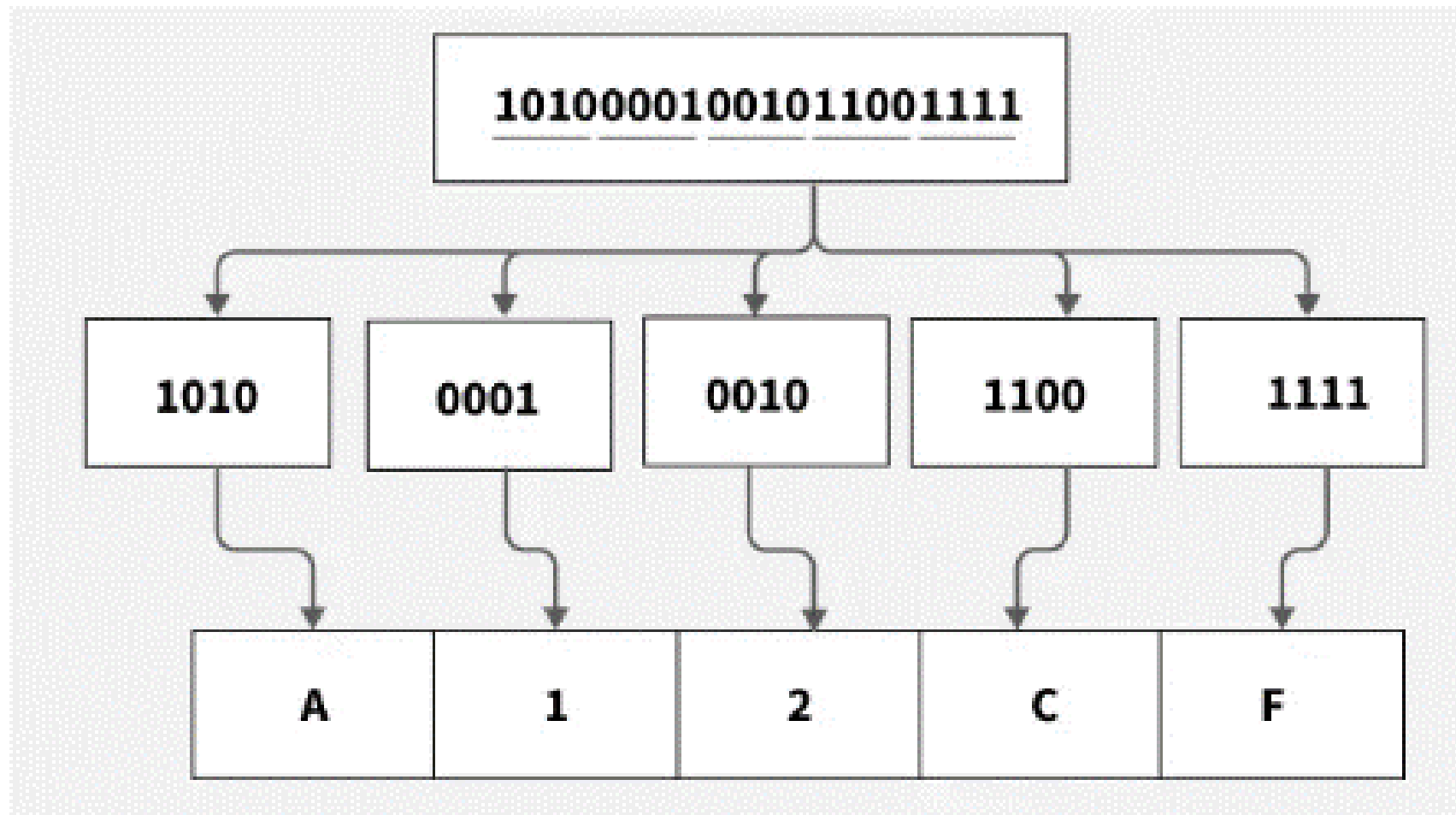
Shortcut Method for Converting a Binary Number to Hexadecimal Number

Grouping Bits

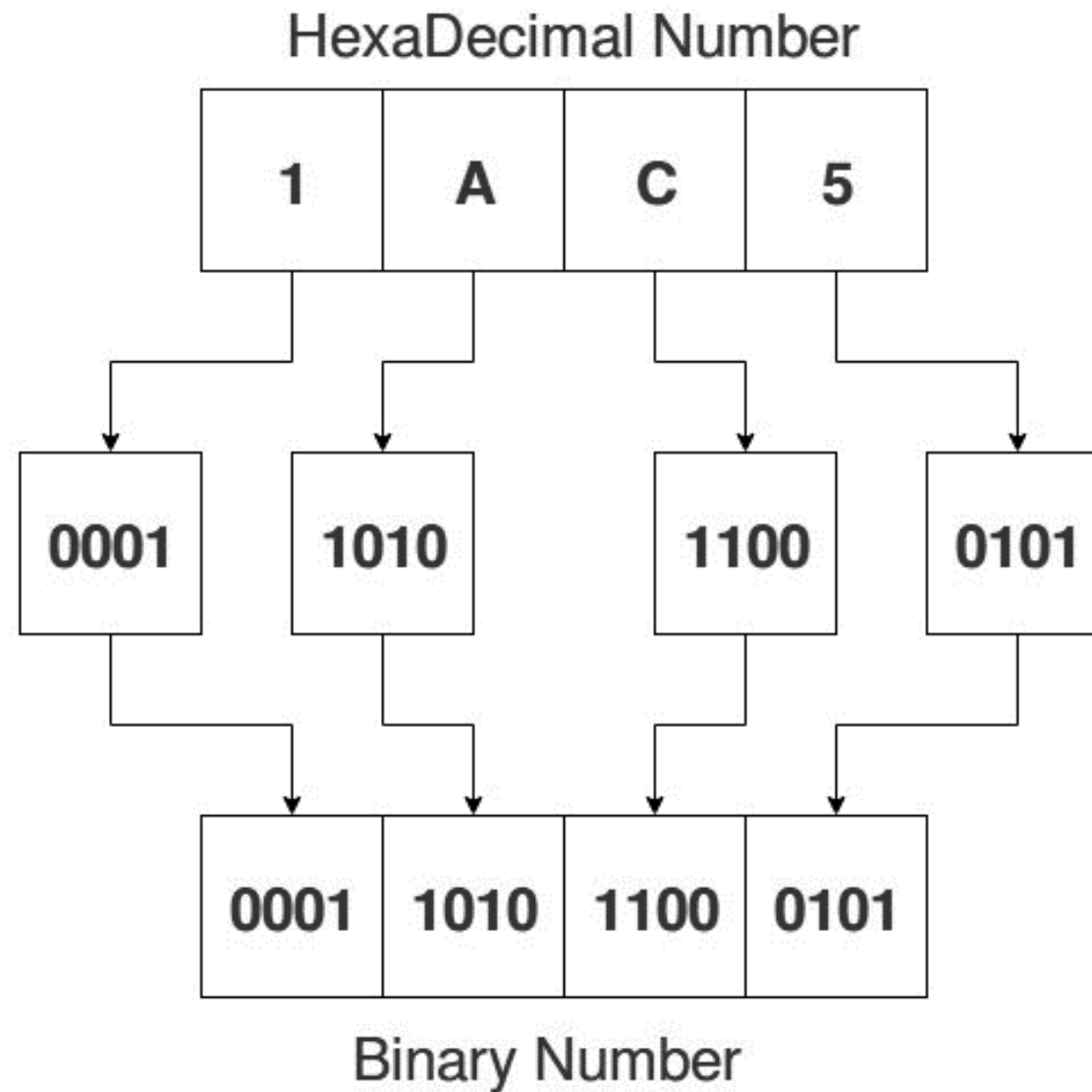
- Step 1: Write the binary number.
- Step 2: Starting from the right (least significant bit), group the binary digits in sets of 4.
- Add extra 0s at the leftmost side if needed to complete a group of 4.
- Step 3: Convert each 4-bit group directly to its hexadecimal equivalent (0–F).
- Step 4: Combine the hexadecimal digits in the same order.

Why it works: $2^4 = 16$, so 4 binary bits map exactly to 1 hex digit.

Binary to Hex



Hex to Binary



Binary Arithmetic

Binary Addition

EXAMPLE

$$\begin{array}{r} 11 \ 01 \ 01 \ 1 \\ 1 \ 0 \ 1 \ 1 \\ \hline 1 \ 0 \ 1 \ 0 \ 0 \end{array} \quad +$$

CHECK THE ANSWER

$$\begin{array}{r} 9 \\ 11 \\ \hline 20 \end{array} \quad +$$

Binary Arithmetic

Binary Addition

Perform the following additions in binary.

- $101_{10} + 40_{10} =$
- $320_{10} + 18_{10} =$
- $76_{10} + 271_{10} =$

Binary Arithmetic

Binary Subtraction

Computer face problem in performing subtraction like; $A-B$

So, it perform the subtraction like this:

$$A+(-B) = A-B$$

This is where complemented binary is necessary.

Complement

Binary Subtraction

Complement are of two types:

- 1's complement

- 2's complement

For binary subtraction, we convert the numbers in binary (0/1), then we take 2's complement of the second number and add it in the first number.

Binary Arithmetic

Binary Subtraction

EXAMPLE 1

Convert $12 - 8$ using two's complement.

Convert to binary

$$12 = 00001100_2$$

$$8 = 00001000_2$$

- Perform one's complement on the 8_{10}

$$00001000_2$$

$$11110111_2$$

Binary Arithmetic

Binary Subtraction

EXAMPLE 1

Perform two's complement.

$$\begin{array}{r} 1\ 1\ 1\ 1\ 0\ 1\ 1\ 1_2 \\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1_2\ ^+ \\ \hline 1\ 1\ 1\ 1\ 1\ 0\ 0\ 0_2 \end{array}$$

Add the two numbers together.

$$= \cancel{1}0\ 0\ 0\ 0\ 0\ 1\ 0\ 0_2 \text{ (Ignore insignificant bits)}$$

Binary Arithmetic

Binary Subtraction

EXAMPLE 2

What happens if the first number is larger than the second?

Try $6_{10} - 10_{10}$

Binary Arithmetic

Binary Subtraction

EXAMPLE 2

Convert to binary

$$6 = 00000110_2$$

$$10 = 00001010_2$$

- Perform one's complement on the 10_{10}

$$00001010_2$$

$$11110101_2$$

Binary Arithmetic

Binary Subtraction

EXAMPLE 2

Add the two numbers together.

$$\begin{array}{r} 00000110_2 \\ 11110110_2 \\ \hline = 11111100_2 \text{ (Ends with a negative bit)} \end{array}$$

Binary Arithmetic

Binary Subtraction

EXAMPLE 2

Perform one's complement on the result

1 1 1 1 1 1 0 0₂

0 0 0 0 0 0 1 1₂

Add 1 to the result.

0 0 0 0 0 0 1 1₂

0 0 0 0 0 0 0 1₂ ⁺

= 0 0 0 0 0 1 0 0₂

We then add the sign bit back.

0 0 0 0 0 1 0 0₂

= 1 0 0 0 0 1 0 0₂

Binary Arithmetic

Binary Subtraction

Perform the following subtractions.

$$22 - 8 =$$

$$76 - 11 =$$

$$6 - 44 =$$

Binary Arithmetic

Binary Multiplication

It follows normal arithmetic rules, which include:

$$0 * 0 = 0$$

$$0 * 1 = 0$$

$$1 * 0 = 0$$

$$1 * 1 = 1$$

Binary Arithmetic

Binary Multiplication

EXAMPLE 1

Complete $15 * 5$ in binary.

Convert to binary

$$15 = 00001111_2$$

$$5 = 00000101_2$$

Ignore any insignificant zeros.

$$\begin{array}{r} \cancel{0000}1111_2 \\ \times \cancel{00000}101_2 \end{array}$$

Binary Arithmetic

Binary Multiplication

EXAMPLE 1

Multiply the first number.

$$\begin{array}{r} 1111_2 \\ 101_2 \times \\ \hline 1111 \end{array}$$

$1111 \times 1 = 1111$

Now this is where the shift and takes place.

Binary Arithmetic

Binary Multiplication

EXAMPLE 1

Shift one place to the left and multiple the third digit.

$$\begin{array}{r} 1111_2 \\ \times 101_2 \\ \hline 1111 \\ 00000 \\ 111100 \end{array}$$

← Shift One Place

$$1111 \times 1 = 1111$$

Binary Arithmetic

Binary Multiplication

EXAMPLE 1

Add the total of all the steps.

$$\begin{array}{r} 1111 \\ 0000 \\ 111100 \\ \hline 1001011 \end{array} +$$

Convert back to decimal to check.

Binary Arithmetic

Calculate the following using binary multiplication shift and add.

- **$12 * 3 = 1\ 0\ 0\ 1\ 0\ 0_2$**
- **$13 * 5 = 1\ 0\ 0\ 0\ 0\ 0\ 1_2$**
- **$97 * 20 = 1\ 1\ 1\ 1\ 0\ 0\ 1\ 0\ 1\ 0\ 0_2$**
- **$121 * 67 = 1\ 1\ 1\ 1\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1_2$**

Binary Arithmetic

Binary Division

It is similar to decimal long division, and uses a shift and subtract method.

Binary Arithmetic

Binary Division

Complete the following: $25/5$

Step 1: Convert both numbers to binary.

$$25 = 1\ 1\ 0\ 0\ 1$$

$$5 = 1\ 0\ 1$$

Step 2: Place the numbers accordingly:

$$1\ 0\ 1 \overline{) 1\ 1\ 0\ 0\ 1}$$

Binary Arithmetic

Binary Division

Step 3: Determine if 1 0 1 (5) will fit into the first bit of dividend.

$$1\ 0\ 1 \overline{) \boxed{1} 1\ 0\ 0\ 1}$$

1 0 1 (5) will not fit into 1 (1)

Step 4: Place a zero above the first bit and try the next bit.

Binary Arithmetic

Binary Division

$$\begin{array}{r} 00101 \\ 101 \overline{) 11001} \\ \underline{011} \\ 00101 \\ \underline{011} \\ 000 \end{array}$$

Binary Arithmetic

Binary Division

Complete the following divisions:

- 40/4
- 36/7

Signed number arithmetic

For positive binary numbers simple binary representation is enough to use in arithmetic.

For negative binary number, 1's and 2's complement of the binary counter parts.

Overflow

Overflow occurs when the result of an arithmetic operation exceeds the range that can be represented with a fixed number of bits.

In signed Binary numbers, overflow can happen during addition or subtraction.

Overflow in 1's complement

It is an ambiguity with two representation of zero.

$$+0 = 0000 \text{ and } -0 = 1111$$

It range from

$$-(2^{n-1} - 1) \text{ to } +(2^{n-1} - 1)$$

For 4 bits

$$-7 \text{ to } +7$$

Overflow occurs when adding two numbers with the same sign and the result has the opposite sign.

Overflow in 1's complement

Example (4-bit 1's complement)

Add +5 (0101) and -4 (1011 – 1's)

Result shows that **Overflow occurred**.

End-around carry (important!)

In 1's complement, if there is a carry out of the MSB, add it back to the LSB.

Overflow in 2's complement

Overflow occurs when adding two numbers with the same sign and the result has the opposite sign.

Two types of overflow:

- Positive Overflow – when 2 positive number produces a negative result.
- Negative Overflow – when two negative number produces a positive result.

The range for 2s complement is -2^{n-1} to $+(2^{n-1} - 1)$

For four bit -8 to $+7$

Overflow in 2's complement

Example

+7 = 0111

+3 = 0011

1010

-5 = 1011

-4 = 1100

0111

Overflow in 2's complement

Example (4-bit 2's complement)

Add +5 (0101) and -4 (1100 – 2's)

Result shows that **Overflow occurred**.

The carry bit is out of the range so it will be discarded.

Overflow in 2's complement

Example (4-bit 1's complement)

Add +5 (0101) and -4 (1100 – 2's)

Add -5 (1011 -2's) and +4(0100)

If the overflow occurred we discard the last bit and if it does not exceed the range, MSB will be taken as negative.

End-around carry (important!)

In 1's complement, if there is a carry out of the MSB, **add it back** to the LSB.

Key difference

Feature	1's Complement	2's Complement
Zero representation	Two (+0, -0)	One
End-around carry	Required	Not used
Overflow rule	Sign mismatch	Sign mismatch
Carry test	Not reliable	Reliable
Hardware use	Rare	Standard

Solution

Increase the Number of Bits (Most Common)

Use more bits so the result fits within the representable range.

Example

4-bit range: -8 to $+7$

8-bit range: -128 to $+127$

Detect Overflow & Generate flag

The system raise an overflow flag, or ignore the result or trigger an exception / error

Used in: CPUs, Microcontrollers, ALUs

Solution

Saturation Arithmetic (Used in DSPs)

Instead of wrapping around, clamp the result to the maximum or minimum value.

Used in: Image processing, Audio processing, Signal processing

Check Before Addition (Software Method)

Before adding: If both numbers are positive, check if result $> \text{max}$

If both numbers are negative, check if result $< \text{min}$

Used in: Embedded systems, Safety-critical software

Solve

Q1. Add using 1's complement (4-bit)

- $(+5) + (+3)$
- $(+6) + (+3)$
- $(+5) + (-2)$
- $(-4) + (-3)$ Check for overflow)

Q2. Add using 2's complement (4-bit)

- $(+4) + (-3)$
- $(+7) + (+2)$
- $(-5) + (-4)$ Check for overflow
- $(+6) + (-2)$

Thank you!